Desarrolladores: Raúl Lopresti, Luciana De Micco, Maximiliano Antonelli.
Facultad de Ingeniería Universidad Nacional de Mar del Plata
Nombre del proyecto: Autómata celular asincrónico programable.

ARCHIVO: miniaca.v
ARCHIVO TESTBENCH: miniaca_tb.v

Descripción funcional

El módulo **miniaca** implementa un autómata celular asíncrono programable compuesto por cinco celdas, cada una con cinco entradas. El diseño es modular y escalable, permitiendo la interconexión con otros módulos para formar arreglos mayores. Cada celda actualiza su estado de forma asíncrona en función de su propio estado y el de sus celdas vecinas. El comportamiento del autómata se define mediante una tabla de reglas configurable de 32 bits, y el estado inicial se establece mediante una tabla de 5 bits. Aunque el autómata opera de manera asíncrona, requiere una señal de reloj únicamente para cargar su configuración.

Puertos: señales de entrada y salida

Nombre	Dirección	Descripción
i_clk	Entrada	Señal de reloj. Sincroniza todas las operaciones internas del módulo.
i_prg	Entrada	Señal de modo. Cuando está en 1, el circuito entra en modo programación; cuando está en 0, entra en modo ejecución.
i_cx[4:0]	Entrada	Bus de 5 bits utilizado como canal de interconexión entre celdas. Durante la programación también transporta las señales de control reset_cfg (i_cx[0]), shift_en (i_cx[1]) y serial_data (i_cx[2])
i_c00	Entrada	Entrada externa específica de la celda 0 (bit menos significativo de su vector de entrada)
i_c01	Entrada	Entrada compartida entre la celda 0 (como segundo bit) y la celda 1 (como primer bit).
i_c43	Entrada	Entrada compartida entre la celda 3 (como quinto bit) y la celda 4 (como cuarto bit).
i_c44	Entrada	Entrada externa específica de la celda 4 (bit más significativo de su vector de entrada).
o_cx[4:0]	Salida	Salidas de las cinco celdas internas.

Modo de funcionamiento:

1) Modo de programación (i_prg = 1):

Se permite la configuración del autómata mediante una interfaz serial. Se puede cargar:

- El estado inicial de las 5 celdas.

- Una tabla de transición (reglas) de 32 bits que define el comportamiento de las celdas.
- La programación se realiza mediante un registro de desplazamiento controlado por las señales reset_cfg, shift_en y serial_data (todos provenientes del bus i_cx), lo que posibilita la configuración dinámica sin necesidad de múltiples señales externas.

2) Modo de ejecución (i_prg = 0):

Cada celda evalúa sus entradas y genera una salida de acuerdo con la tabla de reglas configurada. La entrada de cada celda es un vector de 5 bits que representa el entorno local (vecinos) y se utiliza como índice en la tabla para obtener el nuevo estado.

Estructura de las celdas

Cada celda está conectada a sus vecinas a través del bus i_cx, junto con algunas señales externas (i_c00, i_c01, i_c43, i_c44) que permiten iniciar el procesamiento en los extremos o propagar información desde celdas adyacentes.
El uso de señales compartidas entre celdas permite emular vecindades deslizantes, haciendo que cada celda vea una porción solapada del entorno global.

Escalabilidad y expansión

El diseño está preparado para escalarse horizontalmente:

- Conexión modular:

Al replicar el módulo miniaca y conectar sus salidas (o_cx) a las entradas compartidas (i_cx) de otro módulo, se puede construir fácilmente un autómata de 10, 15, o más celdas.

- Pasaje de información entre bloques:

Las señales i_c00, i_c01, i_c43, i_c44 permiten la conexión entre los extremos de un bloque y el comienzo del siguiente. Esto es útil para implementar topologías como líneas abiertas, anillos o incluso estructuras bidimensionales.

Aplicaciones

Este tipo de arquitectura es adecuada para:

- Simular autómatas celulares (por ejemplo, reglas tipo Wolfram, como la 110 o 30).
- Implementar lógica distribuida o evolutiva.
- Diseñar procesadores reconfigurables a nivel de celda.
- Prototipar sistemas autoorganizados o reactivos.